

Optimal transfer-interval frequency to minimize data loss in BLE network for healthcare service

Jaehyeok Lee*, Seohu Lee[†], Jayun Hyun[‡], Chan-Yong Park*, Minha Choi[§], Won Jeong Shin[¶],
Ji-Ung Park[§] and Tai-Myoung Chung*

*Department of Computer Science and Engineering, Sungkyunkwan University, Suwon, Republic of Korea

[†]Department of Artificial Intelligence, Sungkyunkwan University, Suwon, Republic of Korea

[‡]Hippo T&C Inc., Suwon, Republic of Korea

[§]Department of Plastic and Reconstructive Surgery, Seoul National University College of Medicine,
Seoul National University Boramae Medical Center, Seoul, Republic of Korea

[¶]Department of Biomedical Research Center, Seoul National University Boramae Medical Center, Seoul, Republic of Korea

Abstract—The use of internet of things technology has substantially increased in healthcare. Accordingly, we have developed a digital therapeutics service, *DfuKare*, which contains a smart insole with multiple sensors named SmarTinsole for diabetes and diabetic foot ulcer patients. However, there are still challenges associated with using multiple sensors in digital therapeutics due to the characteristics of wireless communication, such as network connectivity, real-time capability, and data reliability & integrity. To address these problems, we developed a mobile application that communicates with SmarTinsole called Dfu-App. This paper describes the detailed implementation process from the communication perspective in particular. Moreover, we divide the experiment into three situations that are applicable to a real-world environment and then measure the data loss rate by changing the data transfer-interval frequency of SmarTinsole (10 Hz, 20 Hz, and 40 Hz). With 10 Hz, the data loss rate is 0 % in all cases, but in terms of the collected data, the performance is lower than that with either 20 Hz or 40 Hz. Although the results obtained with 20 Hz and 40 Hz show similar performance, we confirm that 20 Hz is the most appropriate data transfer-interval frequency while considering the hardware overhead of 40 Hz to be troublesome. Based on these results, we verify that communication with SmarTinsole and Dfu-App is reliable and secure, and it is expected that the data that can be collected using *DfuKare* will be helpful for building a dataset and designing an artificial intelligence model in healthcare service.

Index Terms—healthcare service, Bluetooth low energy, wireless communication, data reliability, real-time processing

I. INTRODUCTION

Since internet of things (IoT) technology first emerged in the healthcare industry, this technology has played increasingly an important role in society [1]. The convergence of IoT and healthcare technology not only reduces the pain felt by individual members of society but also reduces the burden of medical expenditures by preventing diseases or managing patients [2], [3].

As one of those efforts, we developed *DfuKare* as a digital therapeutic for the foot management service of chronic diabetes patients and diabetic foot ulcer patients. *DfuKare* contains an IoT-based wearable SmarTinsole that senses, collects, and transmits real-time foot pressure, temperature, humidity, and accelerometer data. Dfu-App is a mobile app that collect data from SmarTinsole using Bluetooth low energy

(BLE) communication and sends the data in the internal storage. However, there have been many difficulties involved in exchanging vast amounts of heterogeneous real-time data with mobile devices.

Due to innate technical characteristics and limitations such as communication protocols and operating system design loopholes, many healthcare services have faced problems such as availability, reliability, data overload & accuracy, etc [4]–[6]. To address the existing issues, the implementation of Dfu-App focuses on overcoming the limitations of BLE communication and minimizing data loss while still ensuring data stability.

DfuKare is likely to be exposed to stressful situations in the real world because it is targeted towards chronic elderly patients who may be unfamiliar with using smart devices. Therefore, IoT-based wearable devices and their apps for older adults require thorough pre-testing for verification. Considering the constraints and general requirements for IoT use cases, validating BLE-based applications must meet requirements such as energy efficiency, small memory footprint, support for heterogeneous hardware, network connectivity, interoperability, real-time capability, data reliability & integrity, and security & safety. In this paper, we focused on proving the network connectivity, real-time capability, and data reliability & integrity of Dfu-App [7], [8].

Network connectivity, which is one of the crucial features of IoT, refers to the ability to communicate and interact both locally and globally [9]. To prove network connectivity in real life, we conducted an experiment assuming a situation in which a user may be far away from a mobile device or where obstacles interfere with the communication signals.

Real-time capability not only refers accurate and timely operation but also imply that users can monitor them in real-time [10]. We performed the experiments in different environments by applying multiple sampling rates to demonstrate the real-time capability and the data reliability & integrity. Data reliability & integrity refers to a system's ability to perform necessary tasks under defined conditions and at specified times [11]. In this paper, the meaning of data reliability & integrity also includes medical availability.

In terms of support for heterogeneous hardware, an NRF

TABLE I: Descriptions of sensors in SmarTinsole (Total Sampling Frequency per insole = 40.33Hz)

Sensor Type	Description	# used per insole	Sampling Frequency
Pressure	MF01A-N-221-A06	8	20 Hz
Temperature Humidity	SHT 35	1	0.33Hz
Accelerometer	MPU-6500	1	20 Hz

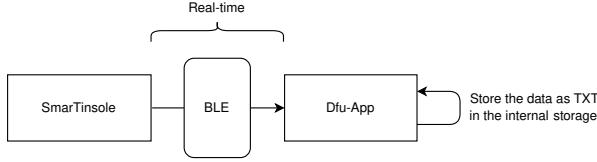


Fig. 1: Overall process of Dfu-App

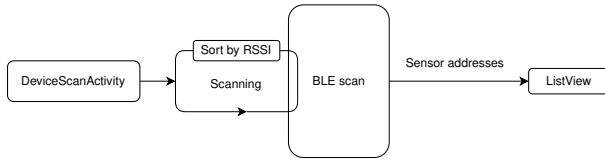


Fig. 2: BLE scanning process

microcontroller (Nordic Semiconductor Inc., Trondheim, Norway) was used in the internal test phase, and it was found to be able to communicate with mobile devices. Moreover, Samsung Galaxy A32 (Samsung Electronics Co., Ltd., Gyeonggi-do, South Korea), an entry-model mobile device, is used. Therefore, it is assumed that the device meets the requirements to a sufficient level. Meanwhile, energy efficiency, small memory footprint, interoperability, and security & safety are not considered in this paper.

This paper describes the development and implementation of Dfu-App, an Android-based application, and reports to the results of performance verification experiments. Section II outlines the process of BLE communication, obtaining data process, and storing data process of the Dfu-App. Sections III and IV present experiments and their results, respectively. Finally, Section V and VI discuss and conclude our implementation, the experimental results, and future works.

II. IMPLEMENTATION

The Dfu-App communicates with SmarTinsole and stores the collected data in internal storage, and it operates through BLE communication with sensors. It stores the pressure, temperature, and humidity data as TXT extension files with a timestamp in the mobile device's internal storage. After connecting the insoles and the application, the data obtained from the sensors are received in the application using the background process. We describe the overall data flow in Fig. 1 and summarize the implementation details based on the data flow. Moreover, Table I lists additional information about the sensors.

This section describes the detailed process of Dfu-App according to the three main functionalities; BLE Communi-

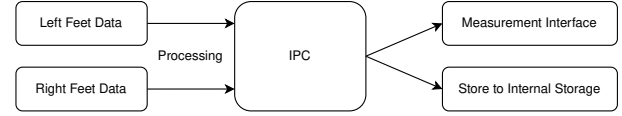


Fig. 3: Obtaining data process

cation, Obtaining Data Process, and Storing Data in Internal Storage.

A. BLE Communication

Let the activity that searches the BLE sensors be referred to as DeviceScanActivity. Before receiving sensor data, the Dfu-App connects to the sensor using Bluetooth. When the application initializes DeviceScanActivity, scanning for BLE sensors is repeated for about 5 minutes.

The scanning method returns the sorted list according to the sensor's received signal strength indicator (RSSI). After receiving the scanned list, we sort the list using the linked hashmap data structure.

Through these processes, the Dfu-App finds two available BLE sensors (both feet). We depict the simplified algorithm used to scan for BLE sensors in Fig. 2.

B. Obtaining Data Process

The Dfu-App receives the addresses of the BLE sensors attempting to connect to it through the aforementioned DeviceScanActivity. While connecting to the BLE sensors, the application delivers the sensors' addresses and the subject's name to the handler thread as parameters. Next, the application starts to obtain the pressure, temperature, humidity, and accelerometer data from the sensors. The algorithm used to obtain multi-sensor data is as follows:

First, the Dfu-App tries to connect to the sensor based on the stored BLE address. After connecting, it starts to collect data according to the callback function at the thread. In addition to the sensor's data, the transmitted data includes information on the collected sensor type, so it performs post-processing by data type.

Second, data is processed for each sensor type to be collected respectively, since there are eight pressure sensors, one temperature and humidity sensor, and one accelerometer sensor per insole. Processed data is sorted and formatted in the following order: collected time, data type, sensor address, and raw data.

Third, the thread delivers the format-matched data to the inter-process communication (IPC). IPC enables communication between components within the same process. The Dfu-



Fig. 4: *DfuKare's* implemented Dfu-App and SmartTinsole

```
20220610_160459.776,1654844699776,20220610_160451.335,1654844691335,6014,0,3152,488,2292,0,0,2,
20220610_160459.783,1654844699783,20220610_160451.385,1654844691385,6534,4648,4624,2004,3430,0,0,450,
20220610_160459.900,1654844699900,20220610_160451.435,1654844691435,6570,6142,3758,804,3684,0,0,330,
20220610_160459.907,1654844699907,20220610_160451.485,1654844691485,6514,6242,4054,2682,4204,0,0,456,
```

Fig. 5: Example of the stored sensor data in TXT file extension

App uses this functionality to collect real-time data from the BLE sensor and then sends it to the measurement interface.

Finally, the fragment represents the data transmitted to visualize the data collection process in real-time.

Through the above four steps, SmartTinsole transfers the data to the Dfu-App. We depict this data collection process in Fig. 3 and an example of SmartTinsole and the application in Fig. 4.

C. Storing Data in Internal Storage

The Dfu-App stores the processed data in its internal storage. If it stores all the data immediately upon collection, the message queue might be stacked with too much data per second; this can result in data loss. To prevent data loss, if the application collects more than five data in obtaining phase, it stores the data as a text file in the internal storage as shown in Fig. 5.

III. DATA LOSS EXPERIMENT

Using medical data leads to more accurate analysis as the higher the frequency, the more sophisticated data can be collected [12], [13]. However, wireless and BLE communication have inevitable obstacles, one of which is data loss. In general, the larger the number of data collected within a set time, the higher the rate of missing data. Because of this, it is essential to identify an *appropriate* transfer frequency. In this case, an *appropriate* frequency refers to the frequency at which a large amount of medically available data can be collected while resulting in negligible loss. Therefore, we conduct experiments while assuming that the following situations apply to the real world.

A. CASE 1 : Without Shoes

Pressure data can represent the user's walking pattern, thus enabling real-time analysis of current walking conditions like swing and stance phases. To accurately categorize the

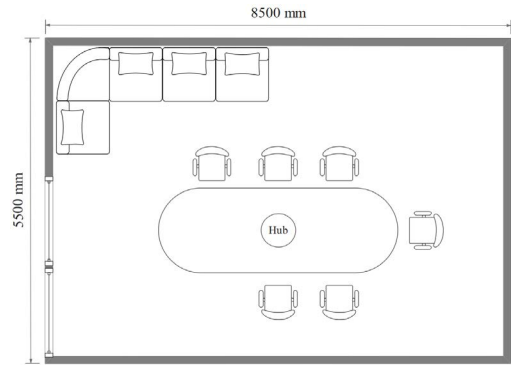


Fig. 6: Structure of a room in which the experiment *Case 3* is conducted

conditions, it is essential to check that data loss does not occur even when pressure is not applied to insoles. Therefore, we take the insoles out of the shoe and collect data for about 10 minutes.

B. CASE 2 : Walking on a straight sidewalk

One of DfuKare's use cases is the management of foot conditions by collecting and analyzing users' multi-sensor data. Therefore, when users walk in shoes equipped with insoles, the process of collecting data must have only a negligible amount of data missing. To confirm this process, a walking experiment is conducted for about 10 minutes on a straight sidewalk. In this experiment, we have each participant hold the Dfu-App in their hand.

C. CASE 3 : Having a smartphone on the center of a room

We will distribute *DfuKare* to the senior care centre, which periodically manages foot conditions in the elderly and prevents and diagnoses diabetic foot ulcers. Therefore, an experiment was conducted by configuring an environment in available homes. Fig. 6 shows a room structure where the experiment is conducted, leaving a smartphone in the center. Moreover, considering that the distance between the smartphone and SmartTinsole is not constant in the real world, we do not change the smartphone's location.

Additionally, we experiment with the abovementioned three situations by changing the data transfer-interval frequency: 10, 20, and 40 Hz. Temperature and humidity sensors collect one data every three seconds, so data loss does not occur. When experimenting using this point, the data loss rate is measured by changing the only frequency of pressure and accelerometer sensors. With these environments, we prove that the Dfu-App has the ability for real-time capability, data reliability, and data integrity.

We aim to distribute *DfuKare* to senior care centers, which periodically manage foot conditions among elderly people and prevents and diagnoses diabetic foot ulcers. Therefore, an experiment was conducted by configuring an environment in available homes. Fig. 6 shows the room structure wherein the experiment is conducted, with a smartphone left in the

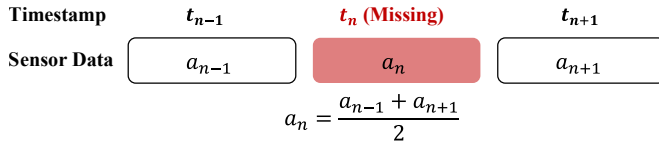


Fig. 7: Linear interpolation for recovering one missing data

center. Moreover, considering that the distance between the smartphone and SmarTinsole is not constant in the real world, we do not change the smartphone's location.

We also experiment with the three situations described above by changing the data transfer-interval frequency among 10, 20, and 40 Hz. Temperature and humidity sensors collect one data point every three seconds, so data loss does not occur. When experimenting using this data, the data loss rate is measured by changing only the frequency of the pressure and accelerometer sensors. In these environments, we prove that the Dfu-App has the features of real-time capability, data reliability, and data integrity.

IV. RESULTS

Assuming that the frequency at which SmarTinsole collects data is α (Hz), one data point is transmitted to the Dfu-App per $\frac{1}{\alpha}$ seconds. If the time gap between the stored data lines is not equal to $\frac{1}{\alpha}$, it can be determined that the data is lost. Further, when the time gap can be represented as $k \cdot \frac{1}{\alpha}$ ($k \geq 2$), $k - 1$ data is lost. Using these calculations, we measure the data loss rates and average them per case as presented in Table II. According to [14], when there is one missing data, the linear interpolation method can restore the original data in a similar manner as shown in Fig. 7. It is therefore necessary to recalculate the loss rate except when only one point of data is lost. Since we measure the data loss rate by comparing the data line intervals, it is clear that the missing data is restorable when the line interval is $\frac{2}{\alpha}$. We recalculate the data loss rate except for the restorable data as shown in Table III. This section summarizes and analyzes the experiment results by frequency and the number of lost data. There are two sections, one case in which one missing data is not restored and one case in which it is recovered, and the experiment results are shown in Figs. 8, 9, and 10.

A. CASE 1 Results

As can be seen in Table II, data loss does not occur when the transfer-interval frequency of SmarTinsole is 10 Hz and 20 Hz. In the case of 40 Hz, data loss is 23 % on average, which means the inssole collects 31 points of data per second. Since the average duration of the stance phase is less than 1 second, the number of collected data at 40 Hz is sufficient to determine that the user is not walking. Therefore, we can confirm that the inssole has properly collected data at all frequencies in III-A.

B. CASE 2 Results

The objective of *DfuKare* is to prevent and manage DFU. Therefore, it is important to verify that SmarTinsole collects

TABLE II: Results of data loss experiments

Experiments	Transfer-interval Frequency	Average Loss Rate	Right foot Loss Rate	Left foot Loss Rate
Case 1	10 Hz	0.000 %	0.000 %	0.000 %
	20 Hz	0.000 %	0.000 %	0.000 %
	40 Hz	23.342 %	25.772 %	20.912 %
Case 2	10 Hz	0.000 %	0.000 %	0.000 %
	20 Hz	0.859 %	1.313 %	0.405 %
	40 Hz	48.007 %	51.439 %	44.474 %
Case 3	10 Hz	0.000 %	0.000 %	0.000 %
	20 Hz	12.391 %	13.299 %	11.482 %
	40 Hz	57.483 %	59.516 %	55.450 %

TABLE III: Recalculated loss rate after interpolation

Experiments	Transfer-interval Frequency	Average Loss Rate	Right foot Loss Rate	Left foot Loss Rate
Case 1	10 Hz	0.000 %	0.000 %	0.000 %
	20 Hz	0.000 %	0.000 %	0.000 %
	40 Hz	3.068 %	3.407 %	2.728 %
Case 2	10 Hz	0.000 %	0.000 %	0.000 %
	20 Hz	0.583 %	0.267 %	0.899 %
	40 Hz	26.057 %	27.698 %	24.415 %
Case 3	10 Hz	0.000 %	0.000 %	0.000 %
	20 Hz	10.368 %	10.597 %	10.139 %
	40 Hz	53.165 %	57.434 %	48.895 %

the data with a low loss rate. 10 Hz shows a loss rate of 0 %, and 20 Hz shows a loss rate of less than 1 %. On the other hand, 40 Hz loses about 19 data per second with a loss rate of 48 %. This means that 20 Hz and 40 Hz collect user data similarly from the perspective of the number of ultimately collected data.

C. CASE 3 Results

When using *DfuKare* in a home such as a Senior Care Center, the distance between the user and the smartphone measuring the user's data (Hub in Fig. 6) might not be constant in all cases. Furthermore, many obstructions, such as various electronic devices and walls, inevitably interfere with communication, thus leading to data loss. Although the data loss rate is equivalent to III-A and III-B at 10 Hz, 20 Hz and 40 Hz show much higher loss rates than the other two cases. The decline in performance can be attributed to interference by surrounding obstacles, and as a result a Dfu-App only collects about 18 of the 20 data. 40 Hz results in a loss rate of 57 %, meaning that there is more than half data loss. About 23 of the 40 data are missing, which can be determined to mean that a similar amount of data was collected to 20 Hz. Therefore, we can confirm that SmarTinsole collects almost the same number of data at 40 Hz and 20 Hz. This proves the network connectivity of inssoles and Dfu-App, confirming that it can be used flexibly in the real world.

D. Results of one missing data recovered

1) *CASE 1 recovered loss Results*: 10 Hz and 20 Hz both show 0 % loss rates in Table II, so they also have the same performance in recovered loss rate. However, 40 Hz

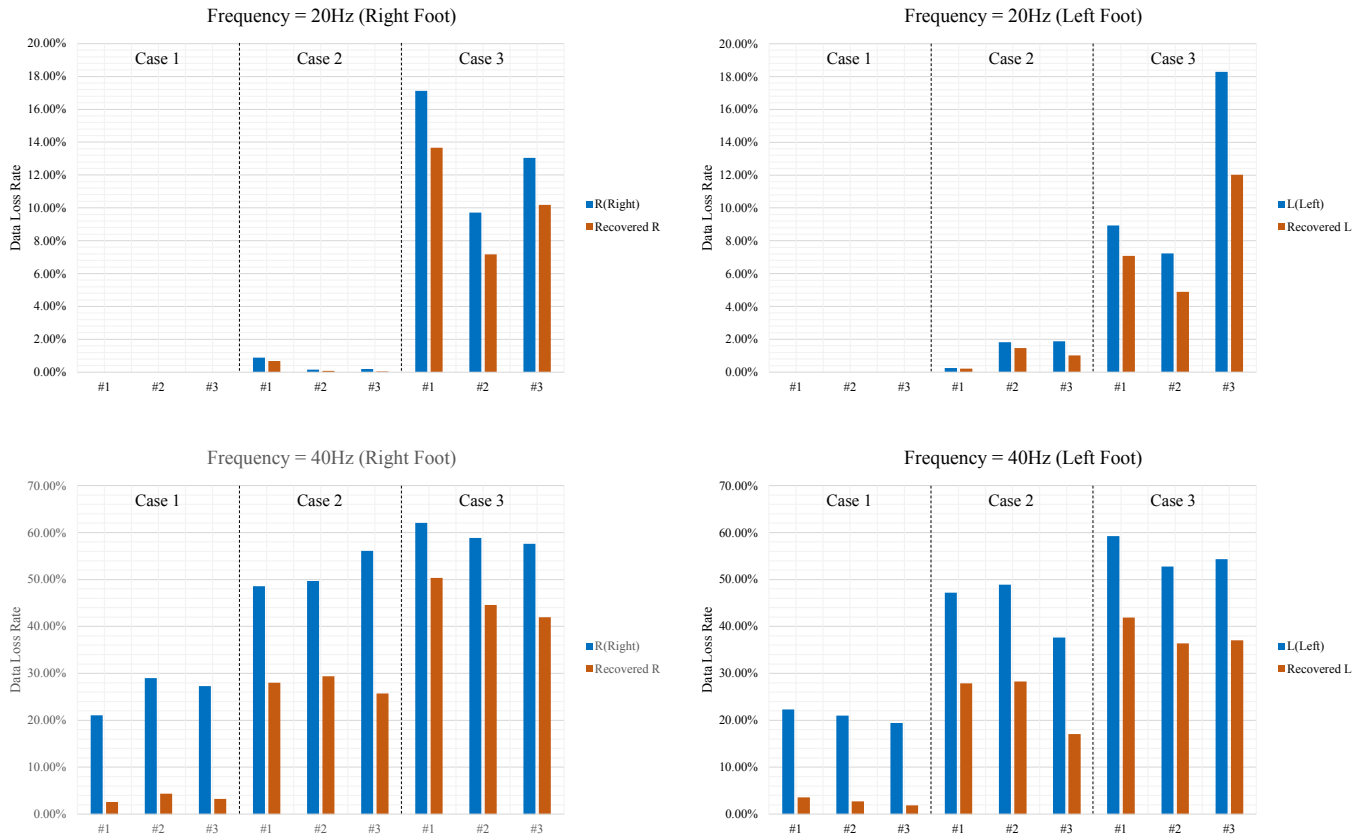


Fig. 8: Graphs showing experiment results of considering the one recovered missing data

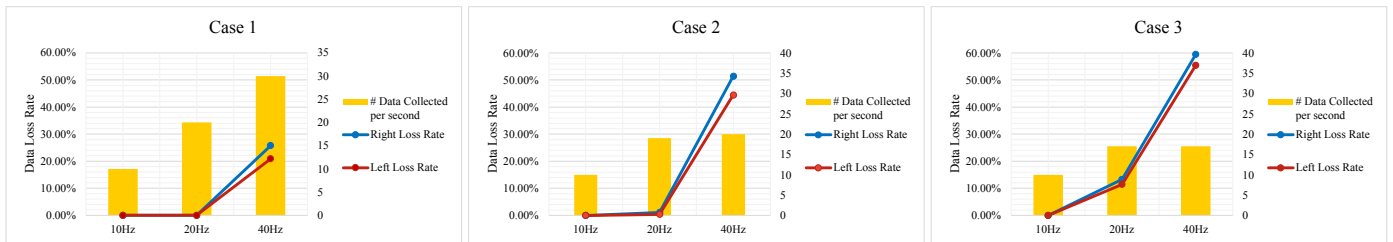


Fig. 9: Two charts showing the unrecovered loss rate and the number of collected data according to the case: Bar charts indicate the number of data collected whereas line charts indicate the loss rate (according to three results from the case-specific experiments)

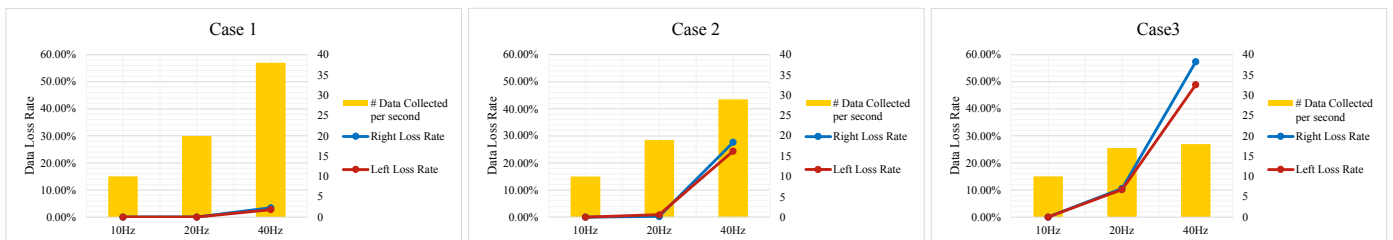


Fig. 10: Graphs showing the recovered loss rate according to the case

outperforms the other frequencies with a 20 % lower loss rate than the not recovered results, and it is confirmed that the data loss in 40 Hz is negligible when without insoles.

2) *CASE 2 recovered loss Results:* When 20 data points are transmitted at a loss rate of 0.5 %, it can be said that Dfu-App collects most of the data. The data loss rate of 40 Hz is reduced by about 22 %, as the application collects about 30 of the 40 data. In this case, the performance of 40 Hz is even better by collecting more data at 40 Hz than at 20 Hz.

3) *CASE 3 recovered loss Results:* The drop of loss rate in 20 Hz and 40 Hz is less than 5 %, and the SmartTinsole collects approximately 20 data at both frequencies. Considering that the drop in loss rate is the most minor compared to the other two cases, it can be said that 40 Hz is not appropriate when users carry out their daily lives with a hub at the center.

V. DISCUSSION

Through the three experiments IV-A, IV-B, and IV-C, we can check that no loss occurs in the case of 10 Hz, and the loss rate performance of 40 Hz is lower than that of 20 Hz. However, from the perspective of the number of data, 10 Hz is the collection of only 10 data per second, which in all cases is less than 20 Hz. Moreover, although 40 Hz transmits 40 data per second, the number of data collected by 20 Hz and Dfu-App may be similar according to the loss rate. Given that the higher the data frequency of the BLE Sensor, the higher the communication overhead cost, the most appropriate frequency for negligible loss rate per second is 20 Hz. This is equivalent to SmartTinsole's original data transfer-interval frequency, which therefore demonstrates that SmartTinsole reliably collects data and communicates with the application. In addition to the results of IV-D, we can also check that both 20 Hz and 40 Hz show similar performance. The higher the frequency, the more overhead is applied to wireless communication, so we can confirm that 20 Hz is much more appropriate than 40 Hz. These results also prove that the application using SmartTinsole at 20 Hz has proper functionalities for network connectivity, real-time capability, and data reliability & integrity.

VI. CONCLUSION

This paper describes the development of a Dfu-App that collects data from multiple sensors in real-time from SmartTinsole. Specifically, we experiment by dividing the three situations corresponding to the use case of *DfuKare*, and the results confirm that the data transfer-interval frequency of the insoles is the most appropriate. In all cases, the loss rate of 10 Hz was 0 %, and the loss rates of 20 Hz and 40 Hz reached up to 12.39 % and 57.48 %, respectively. In addition, when one missing data was restored using the linear interpolation method, the loss rates were 10.37 % for 20 Hz and 53.17 % for 40 Hz. It was found that 20 Hz and 40 Hz collect 1.7 times more data per second than 10 Hz. In particular, an experiment with a smartphone placed in the center of a room showed that 20 Hz and 40 Hz ultimately collect the same number of data per second. Considering the hardware overhead, we can confirm

that 20 Hz is the most appropriate transfer-interval frequency, thus demonstrating the communication reliability with Dfu-App. After collecting walking data using SmartTinsole and Dfu-App to build a dataset, we will design an artificial intelligence model with which to determine the foot condition of diabetic foot ulcer patients and develop service products to enable continuous management of patients.

ACKNOWLEDGMENT

This work was partly supported by Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. 2020-0-00990, Platform Development and Proof of High Trust & Low Latency Processing for Heterogeneous-Atypical-Large Scaled Data in 5G-IoT Environment) and Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (2021-0-01177, Research & Development of Digital Therapeutics for Diabetic Foot Diagnosis and Care).

REFERENCES

- [1] Y. A. Qadri, A. Nauman, Y. B. Zikria, A. V. Vasilakos, and S. W. Kim, "The future of healthcare internet of things: A survey of emerging technologies," *IEEE Communications Surveys Tutorials*, vol. 22, no. 2, pp. 1121–1167, 2020.
- [2] D. A. Gandhi and M. Ghosal, "Intelligent healthcare using iot: a extensive survey," in *2018 second international conference on inventive communication and computational technologies (ICICCT)*. IEEE, 2018, pp. 800–802.
- [3] D. V. Dimitrov, "Medical internet of things and big data in healthcare," *Healthcare informatics research*, vol. 22, no. 3, pp. 156–163, 2016.
- [4] N. Verma, S. Singh, and D. Prasad, "A review on existing iot architecture and communication protocols used in healthcare monitoring system," *Journal of The Institution of Engineers (India): Series B*, pp. 1–13, 2021.
- [5] S. Onofre, P. M. Silvestre, J. P. Pimentão, and P. Sousa, "Surpassing bluetooth low energy limitations on distance determination," in *2016 IEEE International Power Electronics and Motion Control Conference (PEMC)*. IEEE, 2016, pp. 843–847.
- [6] S. Sridhar, P. Misra, G. S. Gill, and J. Warrior, "Cheepsync: a time synchronization service for resource constrained bluetooth le advertisers," *IEEE Communications Magazine*, vol. 54, no. 1, pp. 136–143, 2016.
- [7] Y. B. Zikria, H. Yu, M. K. Afzal, M. H. Rehmani, and O. Hahm, "Internet of things (iot): Operating system, applications and protocols design, and validation techniques," *Future Generation Computer Systems*, vol. 88, pp. 699–706, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X18317710>
- [8] S. Sonune, D. Kalbande, A. Yeole, and S. Oak, "Issues in iot healthcare platforms: A critical study and review," in *2017 International Conference on Intelligent Computing and Control (I2C2)*, 2017, pp. 1–5.
- [9] A. Kulkarni and S. Sathe, "Healthcare applications of the internet of things: A review," *International Journal of Computer Science and Information Technologies*, vol. 5, no. 5, pp. 6229–6232, 2014.
- [10] J. Chanchaichujit, A. Tan, F. Meng, and S. Eaimkhong, "An introduction to healthcare 4.0," in *Healthcare 4.0*. Springer, 2019, pp. 1–15.
- [11] M. H. Kashani, M. Madanipour, M. Nikravan, P. Asghari, and E. Mahdipour, "A systematic review of iot in healthcare: Applications, techniques, and trends," *Journal of Network and Computer Applications*, vol. 192, p. 103164, 2021.
- [12] L. Gao, A. Bourke, and J. Nelson, "Evaluation of accelerometer based multi-sensor versus single-sensor activity recognition systems," *Medical engineering & physics*, vol. 36, no. 6, pp. 779–785, 2014.
- [13] C.-C. Yang, Y.-L. Hsu, K.-S. Shih, and J.-M. Lu, "Real-time gait cycle parameter recognition using a wearable accelerometry system," *Sensors*, vol. 11, no. 8, pp. 7314–7326, 2011.
- [14] V. V. Tipparaju, K. R. Mallires, D. Wang, F. Tsow, and X. Xian, "Mitigation of data packet loss in bluetooth low energy-based wearable healthcare ecosystem," *Biosensors*, vol. 11, no. 10, p. 350, 2021.